

8.2 Binary Trees

A **binary tree** is an ordered tree with the following properties:

1. Every node has at most two children.
2. Each child node is labeled as being either a **left child** or a **right child**.
3. A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of an internal node v is called a **left subtree** or **right subtree**, respectively, of v . A binary tree is **proper** if each node has either zero or two children. Some people also refer to such trees as being **full** binary trees. Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is **improper**.

Example 8.5: An important class of binary trees arises in contexts where we wish to represent a number of different outcomes that can result from answering a series of yes-or-no questions. Each internal node is associated with a question. Starting at the root, we go to the left or right child of the current node, depending on whether the answer to the question is “Yes” or “No.” With each decision, we follow an edge from a parent to a child, eventually tracing a path in the tree from the root to a leaf. Such binary trees are known as **decision trees**, because a leaf position p in such a tree represents a decision of what to do if the questions associated with p ’s ancestors are answered in a way that leads to p . A decision tree is a proper binary tree. Figure 8.5 illustrates a decision tree that provides recommendations to a prospective investor.

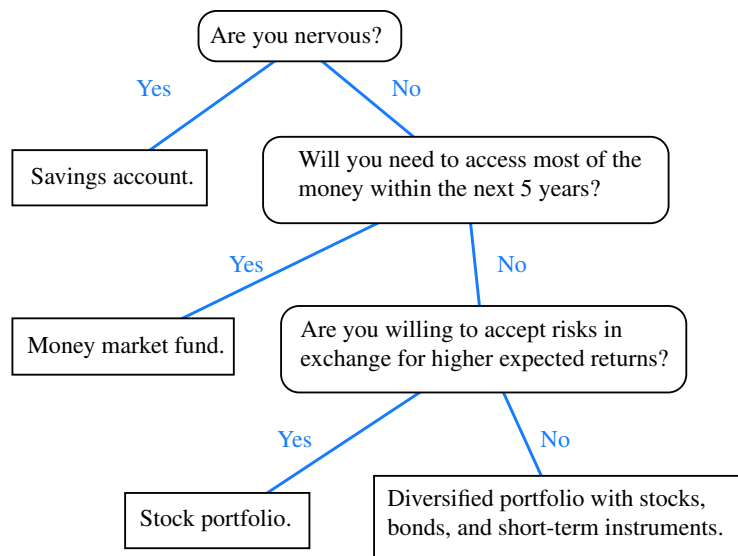


Figure 8.5: A decision tree providing investment advice.

Example 8.6: An arithmetic expression can be represented by a binary tree whose leaves are associated with variables or constants, and whose internal nodes are associated with one of the operators $+$, $-$, $*$, and $/$, as demonstrated in Figure 8.6. Each node in such a tree has a value associated with it.

- If a node is leaf, then its value is that of its variable or constant.
- If a node is internal, then its value is defined by applying its operation to the values of its children.

A typical arithmetic expression tree is a proper binary tree, since each operator $+$, $-$, $*$, and $/$ takes exactly two operands. Of course, if we were to allow unary operators, like negation ($-$), as in “ $-x$,” then we could have an improper binary tree.

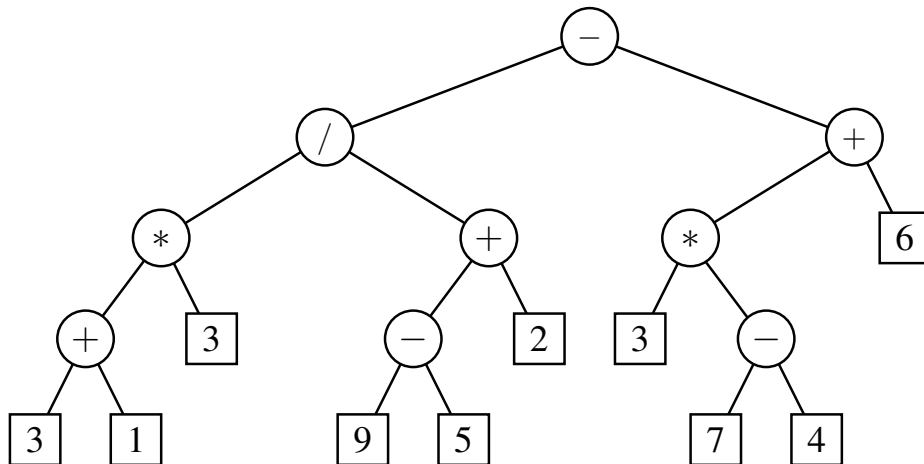


Figure 8.6: A binary tree representing an arithmetic expression. This tree represents the expression $((((3+1)*3)/((9-5)+2)) - ((3*(7-4))+6))$. The value associated with the internal node labeled “/” is 2.

A Recursive Binary Tree Definition

Incidentally, we can also define a binary tree in a recursive way. In that case, a binary tree is either:

- An empty tree.
- A nonempty tree having a root node r , which stores an element, and two binary trees that are respectively the left and right subtrees of r . We note that one or both of those subtrees can be empty by this definition.

8.2.1 The Binary Tree Abstract Data Type

As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

left(*p*): Returns the position of the left child of *p*
(or null if *p* has no left child).

right(*p*): Returns the position of the right child of *p*
(or null if *p* has no right child).

sibling(*p*): Returns the position of the sibling of *p*
(or null if *p* has no sibling).

Just as in Section 8.1.2 for the tree ADT, we do not define specialized update methods for binary trees here. Instead, we will consider some possible update methods when we describe specific implementations and applications of binary trees.

Defining a BinaryTree Interface

Code Fragment 8.6 formalizes the binary tree ADT by defining a `BinaryTree` interface in Java. This interface extends the `Tree` interface that was given in Section 8.1.2 to add the three new behaviors. In this way, a binary tree is expected to support all the functionality that was defined for general trees (e.g., `root`, `isExternal`, `parent`), and the new behaviors `left`, `right`, and `sibling`.

```

1  /** An interface for a binary tree, in which each node has at most two children. */
2  public interface BinaryTree<E> extends Tree<E> {
3      /** Returns the Position of p's left child (or null if no child exists). */
4      Position<E> left(Position<E> p) throws IllegalArgumentException;
5      /** Returns the Position of p's right child (or null if no child exists). */
6      Position<E> right(Position<E> p) throws IllegalArgumentException;
7      /** Returns the Position of p's sibling (or null if no sibling exists). */
8      Position<E> sibling(Position<E> p) throws IllegalArgumentException;
9  }
```

Code Fragment 8.6: A `BinaryTree` interface that extends the `Tree` interface from Code Fragment 8.1.

Defining an AbstractBinaryTree Base Class

We continue our use of abstract base classes to promote greater reusability within our code. The `AbstractBinaryTree` class, presented in Code Fragment 8.7, inherits from the `AbstractTree` class from Section 8.1.2. It provides additional concrete methods that can be derived from the newly declared `left` and `right` methods (which remain abstract).

The new sibling method is derived from a combination of left, right, and parent. Typically, we identify the sibling of a position p as the “other” child of p ’s parent. However, p does not have a sibling if it is the root, or if it is the only child of its parent.

We can also use the presumed left and right methods to provide concrete implementations of the numChildren and children methods, which are part of the original Tree interface. Using the terminology of Section 7.4, the implementation of the children method relies on producing a *snapshot*. We create an empty java.util.ArrayList, which qualifies as being an iterable container, and then add any children that exist, ordered so that a left child is reported before a right child.

```

1  /** An abstract base class providing some functionality of the BinaryTree interface. */
2  public abstract class AbstractBinaryTree<E> extends AbstractTree<E>
3                                     implements BinaryTree<E> {
4      /** Returns the Position of p's sibling (or null if no sibling exists). */
5      public Position<E> sibling(Position<E> p) {
6          Position<E> parent = parent(p);
7          if (parent == null) return null;           // p must be the root
8          if (p == left(parent))                    // p is a left child
9              return right(parent);                // (right child might be null)
10         else                                       // p is a right child
11             return left(parent);                  // (left child might be null)
12     }
13     /** Returns the number of children of Position p. */
14     public int numChildren(Position<E> p) {
15         int count=0;
16         if (left(p) != null)
17             count++;
18         if (right(p) != null)
19             count++;
20         return count;
21     }
22     /** Returns an iterable collection of the Positions representing p's children. */
23     public Iterable<Position<E>> children(Position<E> p) {
24         List<Position<E>> snapshot = new ArrayList<>(2); // max capacity of 2
25         if (left(p) != null)
26             snapshot.add(left(p));
27         if (right(p) != null)
28             snapshot.add(right(p));
29         return snapshot;
30     }
31 }

```

Code Fragment 8.7: An AbstractBinaryTree class that extends the AbstractTree class of Code Fragment 8.2 and implements the BinaryTree interface of Code Fragment 8.6.

8.2.2 Properties of Binary Trees

Binary trees have several interesting properties dealing with relationships between their heights and number of nodes. We denote the set of all nodes of a tree T at the same depth d as **level d** of T . In a binary tree, level 0 has at most one node (the root), level 1 has at most two nodes (the children of the root), level 2 has at most four nodes, and so on. (See Figure 8.7.) In general, level d has at most 2^d nodes.

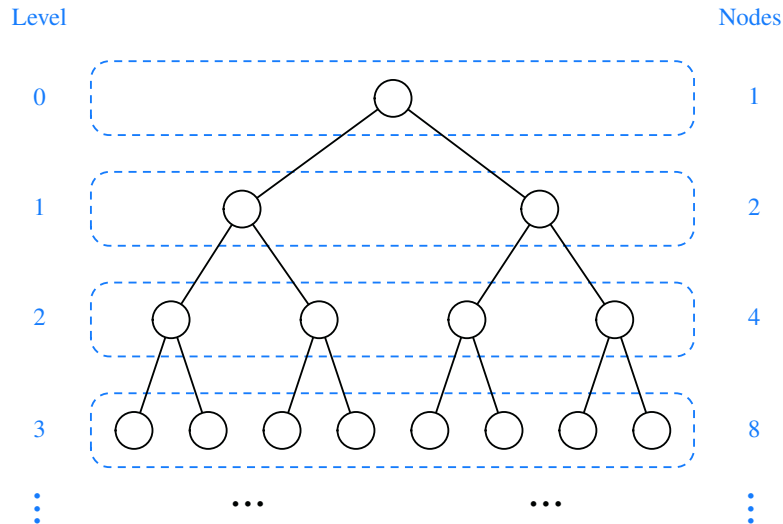


Figure 8.7: Maximum number of nodes in the levels of a binary tree.

We can see that the maximum number of nodes on the levels of a binary tree grows exponentially as we go down the tree. From this simple observation, we can derive the following properties relating the height of a binary tree T with its number of nodes. A detailed justification of these properties is left as Exercise R-8.8.

Proposition 8.7: Let T be a nonempty binary tree, and let n , n_E , n_I , and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T , respectively. Then T has the following properties:

1. $h + 1 \leq n \leq 2^{h+1} - 1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq n - 1$

Also, if T is proper, then T has the following properties:

1. $2h + 1 \leq n \leq 2^{h+1} - 1$
2. $h + 1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq (n - 1)/2$

Relating Internal Nodes to External Nodes in a Proper Binary Tree

In addition to the earlier binary tree properties, the following relationship exists between the number of internal nodes and external nodes in a proper binary tree.

Proposition 8.8: *In a nonempty proper binary tree T , with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.*

Justification: We justify this proposition by removing nodes from T and dividing them up into two “piles,” an internal-node pile and an external-node pile, until T becomes empty. The piles are initially empty. By the end, we will show that the external-node pile has one more node than the internal-node pile. We consider two cases:

Case 1: If T has only one node v , we remove v and place it on the external-node pile. Thus, the external-node pile has one node and the internal-node pile is empty.

Case 2: Otherwise (T has more than one node), we remove from T an (arbitrary) external node w and its parent v , which is an internal node. We place w on the external-node pile and v on the internal-node pile. If v has a parent u , then we reconnect u with the former sibling z of w , as shown in Figure 8.8. This operation, removes one internal node and one external node, and leaves the tree being a proper binary tree.

Repeating this operation, we eventually are left with a final tree consisting of a single node. Note that the same number of external and internal nodes have been removed and placed on their respective piles by the sequence of operations leading to this final tree. Now, we remove the node of the final tree and we place it on the external-node pile. Thus, the external-node pile has one more node than the internal-node pile.

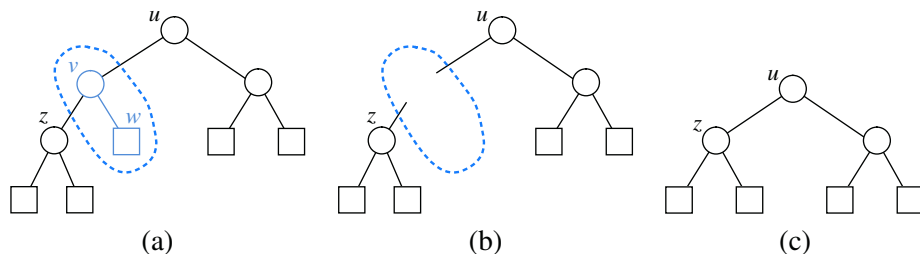


Figure 8.8: Operation that removes an external node and its parent node, used in the justification of Proposition 8.8.

Note that the above relationship does not hold, in general, for improper binary trees and nonbinary trees, although there are other interesting relationships that do hold. (See Exercises C-8.30 through C-8.32.)